DevOps Lead Interview Questions

50 Questions with Detailed Answers · 9 Topic Areas

CI/CD (8)	Containers (7)	Infrastructure (6)
Monitoring (6)	Security (5)	Cloud (5)
Scripting (3)	Culture/Process (5)	Leadership (6)

CI/CD

Q1. What is CI/CD and why does it matter?

ANSWER

CI (Continuous Integration) automatically builds and tests code every time a developer pushes a change. CD (Continuous Delivery/Deployment) automates releasing that tested code to staging or production. Together they reduce manual errors, shorten feedback loops, and let teams ship faster and more reliably.

Q2. What's the difference between Continuous Delivery and Continuous Deployment?

ANSWER

Continuous Delivery ensures every build is deployable but a human approves the production release. Continuous Deployment goes further — every passing build is automatically deployed to production with no manual gate. The choice depends on risk tolerance and compliance requirements.

Q3. How do you handle database migrations in a CI/CD pipeline?

ANSWER

Use migration tools (Flyway, Liquibase, Alembic) version-controlled alongside code. Apply migrations as part of the deployment step, before starting the new app version. Use expand-contract pattern: add new columns before removing old ones, so rollback is safe. Always run migrations in a staging replica first.

Q4. Explain blue-green deployments and when to use them.

ANSWER

You maintain two identical environments — blue (live) and green (idle). New code deploys to green; after testing, traffic switches over. Rollback = flip traffic back to blue. Best for stateless services where you can afford double infrastructure cost. Not ideal when shared databases have schema changes.

Q5. What are canary releases and how do you implement them?

ANSWER

Route a small percentage of traffic (e.g. 5%) to the new version and monitor error rates, latency, and business metrics. Gradually increase traffic if metrics look healthy. Implement via load balancer weights, feature flags, or service mesh traffic splitting (Istio, Argo Rollouts). Automated rollback triggers if thresholds breach.

Q6. How do you secure secrets in a CI/CD pipeline?

ANSWER

Never store secrets in source code or environment variables in plain text. Use secret managers: HashiCorp Vault, AWS Secrets Manager, or GitHub/GitLab CI secret stores. Inject secrets at runtime, rotate them regularly, and audit access. Scan commits for accidental secret leaks with tools like truffleHog or git-secrets.

Q7. What is GitOps and how does it differ from traditional CD?

ANSWER

GitOps uses Git as the single source of truth for infrastructure and app config. An operator (Argo CD, Flux) continuously reconciles the cluster state to match what's in Git. Traditional CD pushes changes imperatively; GitOps pulls them declaratively, making drift detection and rollback trivial — just revert a commit.

Q8. How do you manage pipeline failures across multiple teams?

ANSWER

Establish a broken-build policy (fix within N minutes or revert). Use build ownership so the committer is notified immediately. Maintain a shared pipeline health dashboard. Gate merges to main with required status checks. Treat a red pipeline as a production incident — stop the line, fix it first.

Containers

Q9. Explain the difference between a Docker image and a container.

ANSWER

An image is a read-only template — a layered filesystem snapshot plus metadata. A container is a running instance of an image with a writable layer on top, isolated via Linux namespaces and cgroups. Multiple containers can run from the same image without affecting each other.

Q10. What is Kubernetes and what problems does it solve?

ANSWER

Kubernetes is a container orchestration platform. It solves: scheduling containers on a cluster of nodes, auto-healing crashed containers, scaling workloads up/down, load balancing traffic, managing secrets/config, and rolling out updates with zero downtime. It abstracts away individual server management.

Q11. Explain Kubernetes Deployments vs StatefulSets.

ANSWER

Deployments manage stateless pods — pods are interchangeable, can be replaced in any order, and share no persistent identity. StatefulSets give each pod a stable hostname, persistent volume, and ordered startup/shutdown — essential for databases (Postgres, Kafka, Elasticsearch) that need stable identity.

Q12. How do you handle resource limits in Kubernetes?

ANSWER

Set requests (scheduler uses this for bin-packing) and limits (enforced at runtime) on CPU and memory for every container. Use LimitRange to enforce defaults per namespace. Use ResourceQuota to cap total namespace consumption. Right-sizing requires profiling — tools like Goldilocks (VPA recommender) help automate this.

Q13. What is a Kubernetes Ingress and how does it differ from a Service?

ANSWER

A Service exposes pods within the cluster (ClusterIP) or on each node's port (NodePort) or via a cloud LB (LoadBalancer). An Ingress is an HTTP/HTTPS layer-7 router — it routes external traffic to different Services based on hostname or path rules, with TLS termination. It requires an Ingress controller (nginx, Traefik, ALB).

Q14. How would you debug a pod stuck in CrashLoopBackOff?

ANSWER

Run 'kubectl describe pod ' to read Events. Check logs with 'kubectl logs --previous' to see the last crash output. Common causes: bad config/env vars, missing secrets, failed health checks, OOMKilled (memory limit too low), or app code bug. Use 'kubectl exec' to shell into a working pod for live debugging.

Q15. Explain container image scanning and why it matters.

ANSWER

Image scanning checks the OS packages and language libraries inside a Docker image against CVE databases. Tools: Trivy, Snyk, Grype, Clair. Integrate into CI so builds fail on critical CVEs. Also scan registries continuously — new vulnerabilities are discovered after images are built. Use distroless or minimal base images to reduce attack surface.

Infrastructure

Q16. What is Infrastructure as Code and what are the key benefits?

ANSWER

IaC defines infrastructure (servers, networks, databases) in machine-readable config files checked into version control. Benefits: reproducibility (same code = same environment), auditability (Git history), peer review via PRs, automated drift detection, and disaster recovery (recreate infra from code instead of runbooks).

Q17. Compare Terraform and Ansible for infrastructure management.

ANSWER

Terraform is declarative and excels at provisioning cloud resources (VMs, VPCs, RDS). It manages state and plans changes before applying. Ansible is procedural and excels at configuration management — installing software, managing files, running commands on existing servers. They complement each other: Terraform creates, Ansible configures.

Q18. What is Terraform state and how do you manage it in a team?

ANSWER

State is Terraform's record of what resources it manages and their current attributes. In a team, store state remotely in S3 + DynamoDB (for locking) or Terraform Cloud. Never commit state to Git — it contains sensitive values. Use workspaces or separate state files per environment. State locking prevents concurrent modifications from corrupting it.

Q19. How do you handle Terraform drift?

ANSWER

Drift occurs when someone changes infrastructure outside Terraform (console clicks, manual CLI). Run 'terraform plan' regularly in CI to detect drift — it shows what Terraform would change to reconcile. Tools like Driftctl or Terraform Cloud's continuous validation alert on drift automatically. Fix drift by importing changes or re-applying.

Q20. What is immutable infrastructure and why is it preferred?

ANSWER

Immutable infrastructure means servers are never modified after deployment — instead, you build a new image and replace the old server. Eliminates configuration drift, snowflake servers, and 'it works on my machine' issues. Enables reliable rollback (relaunch old image). Requires good image build pipelines (Packer, Docker) and stateless app design.

Q21. Explain how you would design a highly available multi-region architecture.

ANSWER

Distribute traffic via global load balancer (Route 53 latency routing, CloudFront). Deploy app servers in at least 2 AZs per region, 2+ regions. Use active-active or active-passive DB replication (Aurora Global DB, CockroachDB). Replicate object storage cross-region. Design for failure: circuit breakers, timeouts, retries. Test with chaos engineering.

Monitoring

Q22. What are the four golden signals of monitoring?

ANSWER

Defined by Google's SRE book: Latency (how long requests take), Traffic (request rate), Errors (failure rate), Saturation (how full a resource is — CPU, memory, queue depth). These four signals cover most production issues. Alert on SLO violations, not individual metric thresholds.

Q23. Explain the difference between metrics, logs, and traces.

ANSWER

Metrics are numeric time-series aggregates (CPU %, request count) — cheap to store, great for alerting. Logs are timestamped text events — rich context but expensive at scale. Traces track a request through distributed services showing latency per span — essential for debugging microservice latency. Use all three together (observability pillars).

Q24. What is an SLO and how do you set one?

ANSWER

An SLO (Service Level Objective) is an internal target for reliability, e.g. '99.9% of requests return within 300ms over a 30-day window.' Set them based on user pain (what degradation would they notice?) not on what's easy to achieve. Derive from user journeys, not infrastructure metrics. The gap between 100% and your SLO is your error budget.

Q25. How do you implement distributed tracing?

ANSWER

Instrument services with an OpenTelemetry SDK to emit spans. Propagate trace context (W3C TraceContext header) across service boundaries. Export spans to a backend (Jaeger, Tempo, Zipkin, Datadog). Each span records service name, operation, duration, and tags. Visualize as a flame graph to find bottlenecks. Sample intelligently — 100% tracing is expensive.

Q26. What is the ELK/EFK stack and when would you use it?

ANSWER

ELK = Elasticsearch (search/storage), Logstash (ingestion/transform), Kibana (visualization). EFK replaces Logstash with Fluentd (lighter, Kubernetes-native). Use it for centralized log aggregation, full-text search, and dashboards. At large scale, consider OpenSearch or a managed service (Datadog Logs, Grafana Loki is more cost-effective for logs-only workloads).

Q27. How do you avoid alert fatigue?

ANSWER

Alert only on symptoms that affect users, not every system metric. Use SLO-based alerting (error budget burn rate). Group related alerts. Set meaningful thresholds — no alerts that fire and auto-resolve without action. Route alerts by severity and team. Track alert volume weekly; any alert that fires more than N times without action should be tuned or deleted.

Security

Q28. What is the principle of least privilege and how do you enforce it?

ANSWER

Grant each system/user only the permissions needed for their specific task — nothing more. Enforce via IAM roles (not users) in AWS, RBAC in Kubernetes, service accounts per workload. Audit permissions regularly with tools like IAM Access Analyzer. Use temporary credentials over long-lived keys. Periodically remove unused permissions.

Q29. What is SAST vs DAST?

ANSWER

SAST (Static Application Security Testing) analyzes source code without running it — finds SQL injection patterns, hardcoded secrets, insecure functions. Run in CI on every PR (Semgrep, SonarQube, Checkmarx). DAST (Dynamic) attacks a running application — finds runtime vulnerabilities like XSS, auth bypass. Run against staging (OWASP ZAP, Burp Suite).

Q30. How do you implement network segmentation in a cloud environment?

ANSWER

Use VPCs with public/private subnets. Keep databases in private subnets with no internet route. Use security groups as stateful firewalls per resource. Use NACLs for subnet-level deny rules. Deploy internal services in private subnets accessible only via VPC peering or PrivateLink. Use a NAT gateway for outbound-only internet from private subnets.

Q31. What is mTLS and when would you use it?

ANSWER

Mutual TLS means both client and server present certificates to authenticate each other — not just the server proving its identity. Use in microservices to ensure service-to-service calls are authenticated and encrypted, preventing lateral movement if one service is compromised. Service meshes (Istio, Linkerd) can handle mTLS transparently without app code changes.

Q32. How do you handle a security incident in production?

ANSWER

Immediately contain: isolate affected systems (security group block, revoke credentials). Preserve evidence: snapshot disks, export logs before terminating instances. Assess scope: what data was accessed? Notify stakeholders per your IR plan. Root cause analysis. Remediate and patch. Post-mortem without blame. Update runbooks and detection rules to catch it earlier next time.

Cloud

Q33. Explain the shared responsibility model in cloud.

ANSWER

The cloud provider is responsible for security of the cloud (physical data centers, hypervisor, managed service infrastructure). The customer is responsible for security in the cloud: OS patching on EC2, IAM configuration, data encryption, network rules, app security. Managed services (RDS, Lambda) shift more responsibility to the provider but you still own access control and data.

Q34. What is the difference between vertical and horizontal scaling?

ANSWER

Vertical scaling adds more resources to one server (bigger instance). Simple but has a ceiling, single point of failure, and requires downtime. Horizontal scaling adds more servers. More complex (requires stateless design, load balancing) but unlimited scale potential, no SPOF, and can auto-scale. Design for horizontal from the start.

Q35. How do you choose between containers (ECS/EKS) and serverless (Lambda)?

ANSWER

Use Lambda for event-driven, short-lived, infrequent workloads — zero idle cost, no server management, fast to build. Use containers for long-running services, apps needing persistent connections, complex dependencies, or requiring predictable latency. Lambda cold starts, 15-min timeout, and limited runtime customization are the tradeoffs. Many architectures use both.

Q36. What is a VPC and how does peering work?

ANSWER

A VPC (Virtual Private Cloud) is an isolated network in the cloud where you control IP ranges, subnets, routing, and firewall rules. VPC peering connects two VPCs so resources can communicate privately without going over the internet. Peering is non-transitive — if A peers B and B peers C, A cannot reach C without a direct peering. Use Transit Gateway for hub-and-spoke at scale.

Q37. Explain auto-scaling policies and how to choose between them.

ANSWER

Target tracking: maintain a metric at a target (e.g. 60% CPU) — simplest, recommended for most cases. Step scaling: add/remove different amounts based on how far the metric deviates — more control. Scheduled scaling: scale at known times (e.g. pre-scale before peak hours). Combine scheduled (baseline) with target tracking (dynamic response) for best results.

Scripting

Q38. When would you use Python vs Bash for automation?

ANSWER

Use Bash for simple, short scripts that glue shell commands together — file manipulation, simple pipelines, CI steps. Use Python when you need: data structures, error handling, API calls, parsing JSON/YAML, reusable modules, or anything beyond ~50 lines. Python is more readable, testable, and maintainable at scale. Avoid Bash for anything complex.

Q39. What is idempotency and why does it matter in scripts?

ANSWER

An idempotent operation produces the same result whether run once or many times. Critical in automation: scripts fail partway through; when re-run they shouldn't duplicate resources or fail on 'already exists' errors. Achieve it by checking state before acting (if not exists, create), using tools that are idempotent by design (Terraform, Ansible), or using upsert patterns in databases.

Q40. How do you test infrastructure code?

ANSWER

Unit test modules in isolation (Terratest for Terraform, Molecule for Ansible). Validate syntax with 'terraform validate', 'ansible-lint'. Use policy-as-code tools (OPA, Conftest, Checkov) to enforce standards. Integration test by spinning up real infrastructure in an ephemeral environment. Use contract tests for Terraform module interfaces. Run in CI.

Culture/Process

Q41. What is the three ways of DevOps?

ANSWER

From 'The Phoenix Project': First Way — flow (optimize left-to-right delivery from Dev to Ops to customer; eliminate waste). Second Way — feedback (fast feedback loops right-to-left; find problems early). Third Way — continual learning (culture of experimentation, learning from failure, sharing knowledge). These principles underpin all DevOps practices.

Q42. How do you run a blameless post-mortem?

ANSWER

Focus on systems and processes, not individual mistakes. Document: timeline, contributing factors (5 Whys), impact, and action items with owners/dates. Assume people acted with good intent given what they knew. Publish the post-mortem company-wide — transparency builds trust. Follow up on action items. The goal is learning, not punishment.

Q43. How do you measure DevOps team performance?

ANSWER

Use DORA metrics: Deployment Frequency, Lead Time for Changes, Change Failure Rate, and Mean Time to Recovery. Elite performers deploy multiple times/day with <1h lead time, <5% failure rate, <1h MTTR. These metrics correlate with business outcomes. Avoid vanity metrics (lines of code, ticket count) that don't reflect value delivery.

Q44. How do you balance developer velocity with operational stability?

ANSWER

Use error budgets — when the error budget is spent, slow down deployments until reliability is restored. Implement feature flags to separate deploy from release. Require automated tests before deployment. Run game days to build resilience. Make on-call burden visible to developers so they feel the pain of their reliability decisions. Shared ownership bridges the Dev/Ops gap.

Q45. How would you evangelize DevOps practices in a resistant organization?

ANSWER

Start with a willing team and prove value with metrics. Show business impact (faster feature delivery, fewer incidents). Find executive sponsor. Make the new way easier than the old way — don't add friction. Run internal workshops. Share post-mortems and wins publicly. Avoid big-bang transformations; small, visible wins build momentum. Address fear of job loss directly.

Leadership

Q46. How do you prioritize technical debt vs feature work?

ANSWER

Make tech debt visible — track it in the backlog with estimated business impact. Allocate a fixed percentage of each sprint (20% is common). Link debt to business pain: if it's causing incidents or slowing delivery, it gets prioritized. Avoid declaring debt amnesty (rewrite everything) — pay it down incrementally. Use the 'boy scout rule': leave code better than you found it.

Q47. How do you mentor junior DevOps engineers?

ANSWER

Pair on real problems rather than giving tutorials. Start with well-scoped tasks, expand scope as confidence grows. Review their work with questions ('why did you choose this?') not corrections. Encourage them to own incidents end-to-end with support. Create psychological safety — mistakes in a learning context are expected. Share your own past mistakes to normalize failure.

Q48. Describe how you would onboard a new engineer to a complex platform.

ANSWER

Create a structured onboarding doc covering architecture, key systems, runbooks, and team norms. Have them deploy to staging on day 1 — early wins build confidence. Assign a buddy for the first 30 days. Give a low-risk on-call shadow shift in week 2. Set 30/60/90 day goals. Gather feedback on the onboarding — new engineers see gaps veterans are blind to.

Q49. How do you handle conflict between development and operations teams?

ANSWER

Acknowledge the structural incentive mismatch: Dev is rewarded for speed, Ops for stability. Align incentives — put devs on-call for their own services. Create shared SLOs that both teams own. Facilitate cross-team retrospectives. Make operational requirements (logging, health checks, runbooks) part of the definition of done. Shared pain creates shared investment in solutions.

Q50. How do you build a culture of operational excellence?

ANSWER

Make reliability a first-class feature. Celebrate incident response as much as feature launches. Run game days and fire drills. Make on-call fair and sustainable — rotate, compensate, and track toil. Share post-mortems widely. Set SLOs and review error budgets monthly. Automate toil relentlessly. Leadership must model the behavior: don't skip post-mortems when you're the one who made the mistake.

Q51. What is your approach to capacity planning?

ANSWER

Collect historical growth trends (traffic, storage, compute). Project 6-12 months forward with growth assumptions. Identify bottlenecks before they're hit. Use auto-scaling as the primary buffer but plan for headroom above auto-scale maximums. Include cost in planning — right-size instance families, use reserved/spot capacity. Review quarterly as product roadmap changes demand.
